Practical 2.1: CRD Installation with Docker

Clone, configure, build and launch a complete CRD (GeoNode) instance

Day 2 · 3 hours · Resilience Academy SDI Training

Reference: https://docs.geonode.org/en/master/install/basic/ | Data: Tanga & Mwanza, Tanzania

Learning Objectives:

- Install system prerequisites for GeoNode on Ubuntu 22.04
- Clone and configure the GeoNode project template using `create-envfile.py`
- Build and launch a complete CRD stack using Docker Compose
- Access and administer your local CRD instance
- Compare your local installation to the live crd.resilienceacademy.ac.tz

Prerequisites

- Ubuntu 22.04 LTS (or Docker Desktop on Windows/Mac)
- At least 6GB RAM available to Docker
- 10GB free disk space
- Internet connection for pulling images

Docker memory: On Windows/Mac, open Docker Desktop → Settings → Resources and set memory to at least 6GB before starting.

Step 1: Install System Packages (Ubuntu 22.04)

```
# Install build tools and Python dev packages
sudo add-apt-repository universe
sudo apt-get update -y
sudo apt-get install -y git-core git-buildpackage \
  debhelper devscripts python3.10-dev \
  python3.10-venv virtualenvwrapper

# Install Docker prerequisites
sudo apt-get install -y apt-transport-https \
  ca-certificates curl lsb-release gnupg \
  gnupg-agent software-properties-common vim
```

Step 2: Install Docker Engine

```
# Add Docker GPG key and repository
sudo mkdir -p /etc/apt/keyrings
curl -fsSL \
  https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
```

```
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list \
  > /dev/null

# Install Docker
sudo apt-get update -y
sudo apt-get install -y docker-ce docker-ce-cli \
  containerd.io docker-compose
sudo apt autoremove --purge

# Allow current user to run Docker without sudo
sudo usermod -aG docker ${USER}
su ${USER}
```

Step 3: Create Python Environment and Generate Project

The GeoNode repository is a **Django project template** containing placeholder variables (e.g. `{{project_name}}`). You must use `django-admin startproject` to render the template into a real project before Docker can build it.

```
# Create a Python virtual environment
python3.10 -m venv ~/.venvs/my_crd
source ~/.venvs/my_crd/bin/activate

# Install Django (required for project scaffolding)
pip install Django==4.2.9

# Create destination directory
mkdir ~/my_crd

# Generate project from the GeoNode template (v5.0.1)
GN_VERSION=5.0.1
django-admin startproject \
  --template=https://github.com/GeoNode/geonode-project/archive/refs/heads/$GN_VERSION.
      zip \
  -e py,sh,md,rst,json,yml,ini,env,sample,properties \
  -n monitoring-cron -n Dockerfile \
  my_crd ~/my_crd

cd ~/my_crd
ls -la
```

Alternative method using a local clone:

```
git clone https://github.com/GeoNode/geonode-project.git
cd geonode-project && git checkout 5.0.1
django-admin startproject \
  --template=./geonode-project \
  -e py,sh,md,rst,json,yml,ini,env,sample,properties \
  -n monitoring-cron -n Dockerfile \
  my_crd ~/my_crd
cd ~/my_crd
```

Step 4: Generate Environment Configuration

The rendered project includes `create-envfile.py` to generate a valid `.env` file.

```
# Generate default .env for local development
python create-envfile.py

# Verify .env was created
cat .env | head -20
```

For production deployment with HTTPS:

```
python3 create-envfile.py \
  --https \
  --env_type prod \
  --hostname crd.example.org \
  --email admin@example.org \
  --geonodepwd SecureGeoNodePass \
  --geoserverpwd SecureGeoServerPass \
  --pgpwd SecurePostgresPass \
  --dbpwd SecureDBPass \
  --geodbpwd SecureGeoDBPass
```

Available `create-envfile.py` arguments:

- `-https` — Enable SSL (disabled by default)
- `-env_type` — `prod`, `test`, or `dev`
- `-hostname` — URL serving GeoNode (default: localhost)
- `-email` — Administrator email address
- `-geonodepwd` — GeoNode admin password
- `-geoserverpwd` — GeoServer admin password
- `-pgpwd` — PostgreSQL admin password
- `-dbpwd` — GeoNode database user password
- `-geodbpwd` — GeoNode geo-database user password
- `-clientid` — GeoServer OAuth2 client ID
- `-clientsecret` — GeoServer OAuth2 client secret

Step 5: Build and Start

```
# Build all Docker images (10-20 minutes first time)
docker compose build

# Start all services in background
docker compose up -d

# Monitor Django startup logs
docker compose logs -f django
# Wait for: "[UWSGI] Uwsgi running..."
# Press Ctrl+C to stop following logs

# Check all containers are running
docker compose ps
```

Step 6: Access Your CRD

Default credentials (when using `create-envfile.py` without custom passwords):

- **GeoNode** (`http://localhost/`): username `admin`, password `admin`

- **GeoServer** (`http://localhost/geoserver/`): username `admin`, password `geoserver`

```
# Test that CRD is responding
curl -I http://localhost
# Should return: HTTP/1.1 200 OK
```

Access these URLs in your browser:

- `http://localhost` — Main CRD interface
- `http://localhost/admin/` — Django admin panel
- `http://localhost/geoserver/` — GeoServer admin
- `http://localhost/api/v2/datasets/` — REST API

Compare your local CRD with the live platform: `https://crd.resilienceacademy.ac.tz`

Monitoring and Log Commands

```
# View logs per service
docker compose logs -f django # GeoNode app
docker compose logs -f geoserver # Map server
docker compose logs -f db # Database
docker compose logs -f geonode # NGINX

# Check error log
tail -F -n 300 invoke.log
```

Troubleshooting

```
# Port 80 already in use?
sudo lsof -i :80
# Change HTTP_PORT in .env, then:
docker compose down && docker compose up -d

# GeoServer not connecting?
docker compose logs geoserver | tail -20
docker compose restart geoserver

# Out of memory?
docker stats # see container memory usage
# Increase Docker memory limit to 6GB+

# Full rebuild (drops manual container changes):
docker compose build --no-cache
docker compose up -d
```

Warning: Running `docker compose build -no-cache` drops any manual changes inside containers, except for data in static volumes. Always use the `.env` file for persistent configuration.

Production: HTTPS with Let's Encrypt

For production deployments with automatic SSL certificates:

```
# Generate production .env with HTTPS
python3 create-envfile.py \
```

```
  --https \
  --env_type prod \
  --hostname my-crd.example.org \
  --email admin@example.org

# Key .env settings for production:
# LETSENCRYPT_MODE=production
# HTTP_HOST=
# HTTPS_HOST=my-crd.example.org
# HTTP_PORT=80
# HTTPS_PORT=443

# Rebuild and restart
docker compose build
docker compose up -d
```

Email required: When `LETSENCRYPT_MODE=production`, a valid email address and SMTP server are required for certificate generation.

For custom SSL certificates instead of Let's Encrypt:

```
# Copy certificates into NGINX container
docker exec -it nginx4my_crd_geonode sh -c \
  'mkdir -p /geonode-certificates/my_crd'
docker cp chain.crt \
  nginx4my_crd_geonode:/geonode-certificates/my_crd/
docker cp my_crd.key \
  nginx4my_crd_geonode:/geonode-certificates/my_crd/

# Update NGINX config inside container
docker compose exec geonode sh
# Edit nginx.https.enabled.conf to point to your certs:
# ssl_certificate /geonode-certificates/my_crd/chain.crt;
# ssl_certificate_key /geonode-certificates/my_crd/my_crd.key;
nginx -s reload
exit
```

Checkpoint:

- ✓ `docker compose ps` shows all services as "running"
- ✓ `http://localhost` opens the CRD web interface
- ✓ You can login with admin credentials
- ✓ GeoServer is accessible at `http://localhost/geoserver/`
- ✓ `http://localhost/api/v2/datasets/` returns JSON